RUN TIME PERFORMANCE

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

G1. Feed the Profiler Output Back Into the Prompt

Before asking an AI assistant to speed up code, run a profiler and include the report in your prompt. Tell the model exactly which function is the bottleneck, what percentage of total runtime it consumes, and how many times it is called. Do not ask the model to 'optimize this file' without that data.

Why It Matters

- EffiLearner: per-line profile feedback in a self-optimization loop achieves 87.1% runtime reduction and 90.8% memory reduction (pass@1 drop < 0.5%)
- PerfCodeGen: runtime feedback lets small models rival large models and beats human-written code on ~55% of HumanEval and ~67% of MBPP tasks
- Self-Refine: iterative feedback improves code-optimization scores by an average of 8.7 points on the PIE benchmark
- Without profiler data, models apply syntactic micro-edits uniformly and miss the one function that actually matters
- Models already know efficient patterns but what they lack is the signal to select them

Key Practices: Prompting

- Run cProfile (Python) or perf/gprof/Valgrind (C/C++) before writing the prompt; paste the top 10-20 profiler lines
- State the bottleneck explicitly: function name, % of runtime, total time, and call count
- Specify the input size and access pattern: 'The input list has ~50,000 elements. Called once per request.'
- Ask for one focused change at a time; do not change the function signature or break existing tests
- Ask the model to state the time complexity of its output and compare it to the original

Key Practices: Reviewing

- Re-run the profiler on new code; do not accept the rewrite unless the bottleneck's share of runtime visibly dropped
- Check that the optimized function matches the original on at least five representative inputs before measuring speedup
- If the first rewrite is only syntactic (variable renaming, local lookups), paste the new profile and ask for a fundamentally different algorithm or data structure
- Watch for silent memory-for-time trades (large caches); verify peak memory with tracemalloc or /usr/bin/time -v

G2. Always Verify the Algorithmic Complexity of Generated Code

After receiving any generated or optimized implementation, explicitly ask the model to state and justify the time and space complexity of its output. Then verify that claim by inspecting the loop structure, recursion depth, or data-structure operations. Do not assume a correct solution is also efficient.

Why It Matters

- ENAMEL benchmark: GPT-4 achieves pass@1 of 0.831 but eff@1 of only 0.454 -- half of correct solutions are not competitive on runtime
- Sampling 100 solutions (eff@100) stays at or below 0.483 as the model is not generating better algorithms, just different ones at the same complexity class
- EffiBench: GPT-4 code averages 3.12x human execution time; worst case 13.89x time and 43.92x memory
- 'Sub-Optimal Time Complexity' is one of the most common LLM code defect classes.
- The model's default strategy favors correctness and readability over efficiency; you must ask explicitly for complexity

Key Practices: Prompting

- Add a complexity requirement to the problem statement: 'Your solution must run in O(n log n) time and O(n) space'
- Ask the model to reason about complexity before writing code: 'First, identify the best-known time complexity. Then implement a solution that achieves it.'
- After receiving output, follow up: 'State the time and space complexity and walk through why.'
- For standard algorithmic problems (sorting, DP, graph traversal), name the complexity class you expect and warn against weaker approaches

Key Practices: Reviewing

- Count the nesting depth of loops and recursion as every additional loop nest multiplies complexity by that loop's range
- Check every data structure operation: list.index(), 'in list', list.insert(0,...) are all O(n) in Python and silently make a linear algorithm quadratic
- If the model claims O(n log n), look for the sort, heap, or balanced BST that justifies it and if none is present, the claim is wrong
- Empirical check: time the function on n=1000, 2000, and 4000; if runtime quadruples each time n doubles, the complexity is O(n^2) regardless of claims
- For recursive solutions without memoization, test on a moderately large input (e.g., n=40 for Fibonacci) and watch whether the call count explodes

G3. Provide Typical Input Characteristics to Prompt Algorithm Switching

When the typical size of input to a function differs significantly from the worst case, include that information in your prompt and ask the model to generate a hybrid implementation that selects between two algorithms at a runtime threshold. Do not ask for a single general-purpose algorithm when you know the distribution of inputs in advance.

Why It Matters

- LLMs default to a single general-purpose algorithm and they do not consider whether input size makes a different algorithm faster
- eff@100 stays at or below 0.483 on hard problems but models do not generate hybrid strategies without explicit prompting
- 'Sub-Optimal Time Complexity' recurs because models apply one algorithm uniformly, not based on input properties
- PIE dataset: the most impactful performance edits are algorithmic and they change which algorithm runs, enabled by concrete problem-structure context
- Python's Timsort and C++ introsort both use threshold-based switching and giving an LLM distribution knowledge replicates this expert reasoning

Key Practices: Prompting

- State the typical input size and its frequency explicitly: '90% of calls have n < 32; 10% have n up to 100,000'
- Name the threshold explicitly if you know it, or ask the model to derive one with a constant-factor argument
- Ask for named fast and slow paths, each clearly labeled (e.g., _small_case and _general_case)
- Ask the model to state the complexity of each branch separately and at which input size they cross over
- Specify that both branches must be correct on all inputs and do not add assumptions to the small-case that would fail on large inputs

Key Practices: Reviewing

- Test the small-case branch in isolation: empty input, one element, and an input exactly at the threshold
- Test the general-case branch in isolation on three inputs above the threshold, including a large representative input
- Test at the boundary: n = threshold-1, threshold, and threshold+1; output must match a known-correct reference at all three points
- Measure actual crossover: time both branches across a range of input sizes to confirm the threshold is empirically correct, not just theoretically plausible
- If the model hardcoded a threshold without justification, ask it to show the constant-factor reasoning or run a micro-benchmark

References

[1] Zhu et al. 'More Than Just Functional: LLM-as-a-Critique for Efficient Code Generation.' NeurIPS 2025. openreview.net/forum?id=0Zri6HSYaK

[2] Peng et al. 'PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback.' FORGE@ICSE 2025. arXiv:2412.03578

[3] Huang et al. 'EffiLearner: Enhancing Efficiency of Generated Code via Self-Optimization.' NeurIPS 2024. arXiv:2405.15189

[4] Huang et al. 'EffiBench: Benchmarking the Efficiency of Automatically Generated Code.' NeurIPS 2024. arXiv:2402.02037

[5] Qiu et al. 'How Efficient is LLM-Generated Code? A Rigorous Benchmark (ENAMEL).' NeurIPS 2024. arXiv:2406.06647

[6] Shypula et al. 'Learning Performance-Improving Code Edits.' ICLR 2024. arXiv:2302.07867

[7] Abbassi et al. 'Unveiling Inefficiencies in LLM-Generated Code: Toward a Comprehensive Taxonomy.' ICSME 2025. arXiv:2503.06327

[8] Liu et al. 'What is Wrong with Your Code Generated by Large Language Models?' arXiv:2407.06153, 2024

[9] Madaan et al. 'Self-Refine: Iterative Refinement with Self-Feedback.' NeurIPS 2023. arXiv:2303.17651

References (continued)

[10] Acharya et al. 'Optimizing Code Runtime Performance through Context-Aware RAG.' arXiv:2501.16692, 2025

[11] Gao et al. 'Search-Based LLMs for Code Optimization (SBLLM).' ICSE 2025. arXiv:2408.12159

[12] Taneja et al. 'LLM-Vectorizer: LLM-based Verified Loop Vectorizer.' CGO 2025. arXiv:2406.04693

[13] He et al. 'SimdBench: Benchmarking LLMs for SIMD-Intrinsic Code Generation.' arXiv:2507.15224, 2025

[14] Nichols et al. 'Can Large Language Models Write Parallel Code? (ParEval).' HPDC 2024. arXiv:2401.12554

[15] Huang et al. 'CONCUR: Benchmarking LLMs for Concurrent Code Generation.' arXiv:2603.03683, 2026

[16] Erhabor et al. 'Measuring the Runtime Performance of C++ Code Written by Humans using GitHub Copilot.' arXiv:2305.06439, 2023

[17] Godoy et al. 'An Assessment of LLMs for OpenMP-based Code Parallelization.' Journal of Big Data, 2024. doi:10.1186/s40537-024-01019-z